

Comprehensive Documentation of Tags, Attributes, Classes, and Helper Functions

August 2025

A Manual for dotPipe

Table of Contents

1. [Introduction](#)
2. [Core Concepts](#)
3. [Tags](#)
4. [Attributes](#)
5. [Classes](#)
6. [Helper Functions](#)
7. [Flow and Recursion](#)
8. [Examples](#)
9. [API Reference](#)

Introduction

dotPipe is a JavaScript library designed to simplify DOM manipulation, AJAX operations, and dynamic content loading. It provides a set of custom HTML tags, attributes, and classes that enable developers to create interactive web applications with minimal JavaScript coding.

This manual provides comprehensive documentation of all tags, attributes, classes, and helper functions available in dotPipe, along with examples and explanations of the flow and recursion patterns used in the library.

Core Concepts

dotPipe operates on several key concepts:

1. **Custom Tags:** dotPipe introduces custom HTML tags like `<pipe>`, `<dyn>`, `<lnk>`, etc., that provide specific functionality.
2. **Attributes:** Various attributes can be applied to both custom and standard HTML tags to control behavior.
3. **Classes:** CSS classes are used to trigger specific behaviors or modify how tags operate.
4. **Event Handling:** dotPipe automatically attaches event listeners to elements based on attributes and classes.
5. **AJAX Operations:** The library provides simple ways to make AJAX calls and handle responses.
6. **Content Insertion:** Results from AJAX calls can be inserted into specified elements.
7. **Component System:** dotPipe includes several built-in components like tabs, columns, search, CSV handling, etc.

Tags

<pipe>

The `<pipe>` tag is a core element that initializes on DOMContentLoaded event. It's used for AJAX operations and content insertion.

Required Attributes: - `id`: Unique identifier for the element

Common Attributes: - `ajax`: URL to fetch content from - `query`: Query string parameters - `insert`: ID of the element where content should be inserted - `mode`: HTTP method ("POST" or "GET", default: "POST")

Example:

```
<pipe id="data-loader" ajax="data.json" query="key0:value0;" insert="content-area">
```

<dyn>

The `<dyn>` tag automatically adds event listeners for onclick events. It's similar to `<pipe>` but designed for dynamic interactions.

Required Attributes: - `id`: Unique identifier for the element

Common Attributes: - Same as `<pipe>` tag

Example:

```
<dyn id="button1" ajax="action.php" query="action:click;" insert="result-area">Click me</dyn>
```

<lnk>

The `<lnk>` tag creates clickable links that can perform AJAX operations or navigate to URLs.

Required Attributes: - `id`: Unique identifier for the element

Common Attributes: - `ajax`: URL to navigate to or fetch content from - `query`: Query string parameters

Example:

```
<lnk id="nav-link" ajax="page.html" query="section:home;">Go to Home</lnk>
```

<timed>

The `<timed>` tag refreshes content at regular intervals, keeping information up-to-date.

Required Attributes: - `id`: Unique identifier for the element - `delay`: Time in milliseconds between refreshes

Common Attributes: - `ajax`: URL to fetch content from - `query`: Query string parameters - `insert`: ID of the element where content should be inserted

Example:

```
<timed id="updates" ajax="status.php" delay="3000" insert="status-area"></timed>
```

`<carousel>`

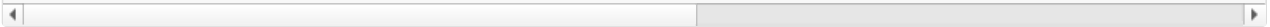
The `<carousel>` tag creates a carousel/slider that can display multiple items and rotate through them.

Required Attributes: - `id`: Unique identifier for the element

Common Attributes: - `ajax`: URL to fetch content from - `file-order`: Semicolon-delimited list of files to display - `delay`: Time in milliseconds between slides - `insert`: ID of the element where content should be inserted - `height`: Height of the carousel - `width`: Width of the carousel - `boxes`: Number of boxes/items to display

Example:

```
<carousel id="image-slider" ajax="images.php" file-order="img1.jpg;img2.jpg;img3.jpg" delay="3000" insert="status-area">
```



`<search>`

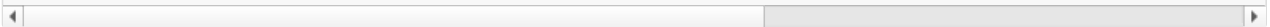
The `<search>` tag creates a search interface that can filter content in specified elements.

Required Attributes: - `id`: Unique identifier for the element

Common Attributes: - `use-id`: Semicolon-delimited list of element IDs to search within - `input-width`: Width of the search input field - `input-height`: Height of the search input field - `placeholder`: Placeholder text for the search input - `search-delay`: Delay in milliseconds before search is performed after typing

Example:

```
<search id="content-search" use-id="content1;content2;content3;" input-width="200px" input-height="30px">
```



`<csv>`

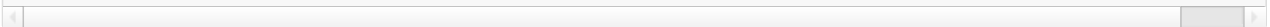
The `<csv>` tag displays CSV data in various formats (table, list, cards).

Required Attributes: - `id`: Unique identifier for the element

Common Attributes: - `sources`: Semicolon-delimited list of CSV file URLs - `csv-as`: Display mode ("table", "list", "cards") - `sort`: Column and direction for sorting - `csv-class`: CSS class to apply to the container - `page-size`: Number of items per page - `lazy-load`: Whether to load data lazily

Example:

```
<csv id="data-table" sources="data1.csv;data2.csv" csv-as="table" sort="name:csv-asc" page-size="10"></csv>
```



<tabs>

The `<tabs>` tag creates a tabbed interface for organizing content.

Required Attributes: - `id`: Unique identifier for the element - `tab`: Semicolon-delimited list of tab definitions in format "TabName:TabId:ContentSource"

Common Attributes: - `class`: CSS classes to apply to tabs - `style`: Inline styles to apply to tabs

Example:

```
<tabs id="content-tabs" tab="Tab1:tab1:tab1.html;Tab2:tab2:tab2.html;Tab3:tab3:tab3.html" class="blue-ta
```

<login>

The `<login>` tag creates login and registration forms.

Required Attributes: - `id`: Unique identifier for the element

Common Attributes: - `login-page`: URL for login form submission - `registration-page`: URL for registration form submission - `css-page`: URL to an external CSS file for styling

Example:

```
<login id="user-auth" login-page="login.php" registration-page="register.php" css-page="auth-styles.css"
```

<columns>

The `<columns>` tag creates a multi-column layout with dynamic content loading.

Required Attributes: - `id`: Unique identifier for the element

Common Attributes: - `count`: Number of columns - `percents`: Comma-separated list of column widths as percentages - `height`: Height of the columns container - `width`: Width of the columns container - `pages`: Semicolon-delimited list of page URLs to load into columns

Example:

```
<columns id="page-layout" count="3" percents="25,50,25" height="500px" width="100%" pages="sidebar.html;
```

<refresh>

The `<refresh>` tag creates a button or mechanism to refresh content in specific targets.

Required Attributes: - `id`: Unique identifier for the element - `target`: Target specification string (format: "targetId:pageUrl;targetId2:pageUrl2")

Common Attributes: - `interval`: Auto-refresh interval in seconds

Example:

```
<refresh id="refresh-btn" target="content:data.html">Refresh Content</refresh>
```

<checkout>

The `<checkout>` tag creates a checkout form for e-commerce applications.

Required Attributes: - `id`: Unique identifier for the element

Common Attributes: - `validate`: Whether to run in validation mode

Example:

```
<checkout id="payment-checkout" validate="true"></checkout>
```

<order-confirmation>

The `<order-confirmation>` tag displays order confirmation details.

Required Attributes: - `id`: Unique identifier for the element

Common Attributes: - `order-id`: ID of the order to display

Example:

```
<order-confirmation id="order-success" order-id="ORD-12345"></order-confirmation>
```

<cart>

The `<cart>` tag creates a shopping cart interface.

Required Attributes: - `id`: Unique identifier for the element

Example:

```
<cart id="shopping-cart"></cart>
```

Attributes

Core Attributes

id

- **Use Case:** Required for all tags to identify the element
- **Example:** `<pipe id="data-loader">`

ajax

- **Use Case:** Specifies the URL to fetch content from
- **Example:** `<pipe id="data-loader" ajax="data.json">`

query

- **Use Case:** Specifies query string parameters for AJAX requests
- **Format:** Key-value pairs separated by colons and semicolons
- **Example:** `<pipe id="data-loader" ajax="data.json" query="key0:value0;key1:value1;">`

insert

- **Use Case:** Specifies the ID of the element where content should be inserted
- **Example:** `<pipe id="data-loader" ajax="data.json" insert="content-area">`

mode

- **Use Case:** Specifies the HTTP method for AJAX requests
- **Values:** "POST" or "GET" (default: "POST")
- **Example:** `<pipe id="data-loader" mode="GET" ajax="data.json" insert="content-area">`

callback

- **Use Case:** Specifies a JavaScript function to call after AJAX request
- **Example:** `<pipe id="data-loader" callback="processData" ajax="data.json">`

callback-class

- **Use Case:** Specifies classes to be used in the callback function
- **Example:** `<pipe id="data-loader" callback="processData" callback-class="data-processor" ajax="data.json">`

headers

- **Use Case:** Specifies HTTP headers for AJAX requests
- **Format:** Key-value pairs separated by colons and ampersands
- **Example:** `<pipe id="data-loader" ajax="data.json" headers="Content-Type:application/json&Authorization:Bearer token">`

form-class

- **Use Case:** Specifies the class of form elements to include in AJAX requests
- **Example:** `<pipe id="form-submit" form-class="user-form" ajax="submit.php">`

event

- **Use Case:** Specifies events that trigger the element's action
- **Format:** Event names separated by semicolons
- **Example:** `<dyn id="hover-element" event="mouseover;mouseleave" ajax="hover.php">`

delay

- **Use Case:** Specifies the delay in milliseconds for timed elements
- **Example:** `<timed id="updates" ajax="status.php" delay="3000" insert="status-area">`

file-order

- **Use Case:** Specifies a list of files to cycle through
- **Format:** File paths separated by semicolons
- **Example:** `<pipe id="image-cycler" file-order="img1.jpg;img2.jpg;img3.jpg" insert="image-container">`

file-index

- **Use Case:** Specifies the current index in the file-order list
- **Example:** `<pipe id="image-cycler" file-order="img1.jpg;img2.jpg;img3.jpg" file-index="1" insert="image-container">`

interval

- **Use Case:** Specifies the number of steps to take when stepping through file-order
- **Example:** `<pipe id="image-cycler" file-order="img1.jpg;img2.jpg;img3.jpg" interval="2" insert="image-container">`

set

- **Use Case:** Sets the value of an element attribute
- **Format:** "element-id:attribute-name:value"
- **Example:** `<dyn id="button" set="target-element:class:active">`

get

- **Use Case:** Gets the value of an element attribute and returns it to this element
- **Format:** "element-id:attribute-name"
- **Example:** `<dyn id="copy-button" get="source-element:data-value">`

delete

- **Use Case:** Deletes the value of an element attribute
- **Format:** "element-id:attribute-name"
- **Example:** `<dyn id="clear-button" delete="target-element:data-value">`

x-toggle

- **Use Case:** Toggles CSS classes on elements
- **Format:** "element-id:class-name" pairs separated by semicolons
- **Example:** `<dyn id="toggle-button" x-toggle="panel1:visible;panel2:hidden">`

display

- **Use Case:** Toggles visibility of elements
- **Format:** Element IDs separated by semicolons
- **Example:** `<dyn id="show-hide-button" display="panel1;panel2;">`

remove

- **Use Case:** Removes elements from the DOM
- **Format:** Element IDs separated by semicolons
- **Example:** `<dyn id="delete-button" remove="temp1;temp2;">`

tool-tip

- **Use Case:** Creates a tooltip for the element
- **Format:** "tooltip-text;id;class;duration;zIndex"
- **Example:** `<dyn id="help-button" tool-tip="Click for help;tooltip1;tooltip;3000;100">`

modal-tip

- **Use Case:** Creates a modal tooltip from a JSON file
- **Format:** "filename.json;duration;zIndex"
- **Example:** `<dyn id="info-button" modal-tip="info.json;3000;100">`

copy

- **Use Case:** Copies the value of an element to the clipboard
- **Example:** `<dyn id="copy-button" copy="text-content">`

file

- **Use Case:** Specifies a filename to download
- **Example:** `<dyn id="download-button" class="download" file="document.pdf">`

directory

- **Use Case:** Specifies the directory path for the file to download
- **Example:** `<dyn id="download-button" class="download" file="document.pdf" directory="/files/">`

turn

- **Use Case:** Cycles through elements in sequence
- **Format:** Element IDs separated by semicolons
- **Example:** `<dyn id="cycle-button" turn="elem1;elem2;elem3;" index="0">`

boxes

- **Use Case:** Specifies the number of boxes/items for carousel elements
- **Example:** `<carousel id="image-slider" boxes="3">`

options

- **Use Case:** Specifies options for select elements
- **Format:** "label:value" pairs separated by semicolons
- **Example:** `<select id="dropdown" options="Option 1:value1;Option 2:value2;">`

Classes

Action Classes

pipe-active

- **Use Case:** Indicates that a pipe element has been activated
- **Example:** `<pipe id="active-pipe" class="pipe-active">`

disabled

- **Use Case:** Disables an element from being triggered
- **Example:** `<dyn id="disabled-button" class="disabled">`

time-active

- **Use Case:** Activates timers for continuous operations
- **Example:** `<timed id="active-updates" class="time-active" delay="3000">`

time-inactive

- **Use Case:** Deactivates timers for continuous operations
- **Example:** `<timed id="paused-updates" class="time-inactive" delay="3000">`

redirect

- **Use Case:** Follows the AJAX call in POST or GET mode
- **Example:** `<pipe id="redirect-link" class="redirect" ajax="page.html">`

download

- **Use Case:** Triggers file download
- **Example:** `<dyn id="download-button" class="download" file="document.pdf">`

clear-node

- **Use Case:** Clears the content of specified nodes
- **Example:** `<dyn id="clear-button" class="clear-node" insert="panel1;panel2;">`

incrIndex

- **Use Case:** Increments through index of file-order
- **Example:** `<pipe id="next-image" class="incrIndex" file-order="img1.jpg;img2.jpg;img3.jpg">`

decrIndex

- **Use Case:** Decrements through index of file-order
- **Example:** `<pipe id="prev-image" class="decrIndex" file-order="img1.jpg;img2.jpg;img3.jpg">`

carousel-step-right

- **Use Case:** Moves the carousel one step to the right
- **Example:** `<dyn id="next-button" class="carousel-step-right" insert="carousel1">`

carousel-step-left

- **Use Case:** Moves the carousel one step to the left
- **Example:** `<dyn id="prev-button" class="carousel-step-left" insert="carousel1">`

carousel-slide-right

- **Use Case:** Continuously slides the carousel to the right
- **Example:** `<dyn id="auto-right" class="carousel-slide-right" insert="carousel1">`

carousel-slide-left

- **Use Case:** Continuously slides the carousel to the left
- **Example:** `<dyn id="auto-left" class="carousel-slide-left" insert="carousel1">`

turn-auto

- **Use Case:** Automatically cycles through carousel items
- **Example:** `<carousel id="auto-carousel" class="turn-auto">`

mouse

- **Use Case:** Enables mouse event handling for the element
- **Example:** `<div id="hover-area" class="mouse" event="mouseover;mouseleave">`

mouse-insert

- **Use Case:** Inserts content on mouse events
- **Example:** `<div id="hover-content" class="mouse-insert" event="mouseover">`

action-class

- **Use Case:** Designates elements that listen to other tags' actions
- **Example:** `<div id="listener" class="action-class">`

multiple

- **Use Case:** Indicates that an element has multiple key/value pairs
- **Example:** `<select id="multi-select" class="multiple">`

Content Display Classes

plain-text

- **Use Case:** Returns plain text to the insertion point
- **Example:** `<pipe id="text-loader" class="plain-text" ajax="content.txt" insert="text-area">`

plain-html

- **Use Case:** Returns content as true HTML
- **Example:** `<pipe id="html-loader" class="plain-html" ajax="content.html" insert="html-area">`

json

- **Use Case:** Processes and displays JSON data
- **Example:** `<pipe id="json-loader" class="json" ajax="data.json" insert="json-display">`

strict-json

- **Use Case:** Returns only JSON to full page as response
- **Example:** `<pipe id="api-call" class="strict-json" ajax="api.php">`

modala

- **Use Case:** Processes JSON data using the modala function
- **Example:** `<pipe id="modal-loader" class="modala" ajax="modal.json" insert="modal-container">`

modala-multi-last

- **Use Case:** Creates multi-ajax calls, removing the last item when limit is reached
- **Example:** `<pipe id="feed-loader" class="modala modala-multi-last" ajax="feed.json:feed-container:10" insert="feed-container">`

modala-multi-first

- **Use Case:** Creates multi-ajax calls, removing the first item when limit is reached
- **Example:** `<pipe id="feed-loader" class="modala modala-multi-first" ajax="feed.json:feed-container:10" insert="feed-container">`

tree-view

- **Use Case:** Displays hierarchical data as a tree
- **Example:** `<pipe id="tree-loader" class="tree-view" ajax="tree.json" insert="tree-container">`

Helper Functions

Core Functions

```
domContentLoaded(again = false)
```

- **Purpose:** Initializes all dotPipe elements when the DOM is loaded
- **Parameters:**
- `again`: Boolean indicating whether to re-initialize elements
- **Returns:** void

```
addPipe(elem = document)
```

- **Purpose:** Attaches global event listeners to the document
- **Parameters:**
- `elem`: The element to attach listeners to (default: document)
- **Returns:** void

```
pipes(elem, stop = false)
```

- **Purpose:** Core function that processes elements and their attributes
- **Parameters:**
- `elem`: The element to process
- `stop`: Boolean to stop processing
- **Returns:** void

```
navigate(elem, opts = null, query = "", classname = "")
```

- **Purpose:** Handles AJAX navigation and content loading
- **Parameters:**
- `elem`: The element triggering navigation
- `opts`: AJAX options
- `query`: Query string
- `classname`: Form class name
- **Returns:** void

Modal and Content Functions

```
modal(filename, tagId)
```

- **Purpose:** Displays a modal dialog with content from a JSON file
- **Parameters:**
- `filename`: URL or path to the JSON file
- `tagId`: ID of the element to insert the modal into
- **Returns:** Promise

```
modallist(filenamees)
```

- **Purpose:** Displays multiple modals from JSON files
- **Parameters:**
- `filenamees`: Semicolon-separated list of filenames and targets
- **Returns:** void

```
modala(value, tempTag, root, id)
```

- **Purpose:** Recursively creates HTML elements from a JSON object
- **Parameters:**
 - `value` : JSON object with element definitions
 - `tempTag` : Target element to append to
 - `root` : Root element
 - `id` : Element ID
- **Returns:** Element

```
modalaHead(value)
```

- **Purpose:** Creates HTML elements in the document head from a JSON object
- **Parameters:**
 - `value` : JSON object with element definitions
- **Returns:** void

```
textCard(text, id = "", classes = "", x_center = false, y_center = false, duration = -1, zIndex = 100)
```

- **Purpose:** Creates a text card overlay
- **Parameters:**
 - `text` : Text content
 - `id` : Element ID
 - `classes` : CSS classes
 - `x_center` : X position or centering
 - `y_center` : Y position or centering
 - `duration` : Display duration in milliseconds
 - `zindex` : Z-index value
- **Returns:** void

```
modalCard(filename, x_center = false, y_center = false, duration = -1, zIndex = 100)
```

- **Purpose:** Creates a modal card overlay from a JSON file
- **Parameters:**
 - `filename` : JSON file path
 - `x_center` : X position or centering
 - `y_center` : Y position or centering
 - `duration` : Display duration in milliseconds
 - `zindex` : Z-index value
- **Returns:** void

Data Functions

```
getJSONFile(filename)
```

- **Purpose:** Fetches and parses a JSON file
- **Parameters:**
 - `filename` : URL or path to the JSON file

- **Returns:** Promise with parsed JSON

`getTextFile(filename)`

- **Purpose:** Fetches a text file
- **Parameters:**
 - `filename`: URL or path to the text file
- **Returns:** Promise with text content

`formAJAX(elem, classname)`

- **Purpose:** Collects form data for AJAX requests
- **Parameters:**
 - `elem`: Form element
 - `classname`: Class of form elements to include
- **Returns:** Query string

`setAJAXOpts(elem, opts)`

- **Purpose:** Sets AJAX request options
- **Parameters:**
 - `elem`: Element with AJAX attributes
 - `opts`: Options map
- **Returns:** Updated options map

UI and Display Functions

`renderTree(value, tempTag)`

- **Purpose:** Renders a tree view from a JSON object
- **Parameters:**
 - `value`: JSON object with tree structure
 - `tempTag`: Target element or ID
- **Returns:** Rendered element

`setTimers(target)`

- **Purpose:** Sets up timers for elements that need periodic updates
- **Parameters:**
 - `target`: Element to refresh
- **Returns:** void

`carousel(elem, auto = false)`

- **Purpose:** Initializes a carousel element
- **Parameters:**
 - `elem`: Carousel element
 - `auto`: Whether to auto-rotate

- **Returns:** void

```
shiftFilesLeft(elem, auto = false, delay = 1000)
```

- **Purpose:** Shifts carousel items to the left
- **Parameters:**
 - `elem`: Carousel element
 - `auto`: Whether to continue automatically
 - `delay`: Delay between shifts
- **Returns:** void

```
shiftFilesRight(elem, auto = false, delay = 1000)
```

- **Purpose:** Shifts carousel items to the right
- **Parameters:**
 - `elem`: Carousel element
 - `auto`: Whether to continue automatically
 - `delay`: Delay between shifts
- **Returns:** void

```
fileOrder(elem)
```

- **Purpose:** Manages file ordering for elements with file-order attribute
- **Parameters:**
 - `elem`: Element with file-order attribute
- **Returns:** void

```
copyContentById(id)
```

- **Purpose:** Copies content of an element to clipboard
- **Parameters:**
 - `id`: ID of element to copy
- **Returns:** Boolean success indicator

```
prettyifyJsonWithColors(jsonObj)
```

- **Purpose:** Formats JSON with syntax highlighting
- **Parameters:**
 - `jsonObj`: JSON object to format
- **Returns:** HTML string with colored JSON

```
displayColoredJson(elementId, jsonObj)
```

- **Purpose:** Displays colored JSON in an element
- **Parameters:**
 - `elementId`: Target element ID
 - `jsonObj`: JSON object to display
- **Returns:** void

Flow and Recursion

dotPipe uses several flow and recursion patterns to handle dynamic content and nested structures:

Event Flow

1. Initialization Flow:

2. The library initializes when the DOM content is loaded
3. `DOMContentLoaded` event triggers `domContentLoaded()`
4. `domContentLoaded()` processes all custom tags
5. Event listeners are attached to elements

6. Interaction Flow:

7. User interactions trigger event listeners
8. Event listeners call the `pipes()` function
9. `pipes()` processes the element's attributes and classes
10. Actions are performed based on attributes and classes

11. AJAX Flow:

12. AJAX requests are initiated by `navigate()`
13. Responses are processed based on element classes
14. Content is inserted into target elements
15. Additional processing may occur based on content type

Recursion Patterns

1. DOM Traversal Recursion:

2. `addPipe()` recursively traverses the DOM to find elements
3. Event listeners are attached to elements during traversal

4. JSON to DOM Recursion:

5. `modala()` recursively processes JSON objects
6. Each object property may represent a nested element
7. Elements are created and appended to the DOM

8. Tree View Recursion:

9. `renderTree()` recursively builds tree structures
10. Each node may contain child nodes
11. Event listeners are attached to tree nodes

12. Component Initialization Recursion:

13. Component initialization functions recursively process their elements
14. Child elements are initialized after parent elements
15. Event listeners are attached to component elements

Examples

Basic AJAX Example

```
<!-- Load content from data.json into the content-area element -->
<pipe id="data-loader" ajax="data.json" insert="content-area"></pipe>

<!-- Display the loaded content -->
<div id="content-area"></div>
```

Form Submission Example

```
<!-- Form with AJAX submission -->
<form id="user-form">
  <input type="text" name="username" class="user-form-class" placeholder="Username">
  <input type="password" name="password" class="user-form-class" placeholder="Password">

  <!-- Submit button using pipe -->
  <pipe id="submit-button" form-class="user-form-class" ajax="login.php" insert="result" class="button">
    Login
  </pipe>

  <!-- Result area -->
  <div id="result"></div>
</form>
```

Tabbed Interface Example

```
<!-- Create a tabbed interface -->
<tabs id="content-tabs" tab="Home:tab1:home.html;Products:tab2:products.html;Contact:tab3:contact.html">
```

Carousel Example

```
<!-- Create an image carousel -->
<carousel id="image-slider" ajax="images.php" file-order="img1.jpg;img2.jpg;img3.jpg" delay="3000" insert="image-slider">

<!-- Carousel navigation buttons -->
<div class="carousel-controls">
  <dyn id="prev-button" class="carousel-step-left" insert="image-slider">Previous</dyn>
  <dyn id="next-button" class="carousel-step-right" insert="image-slider">Next</dyn>
  <dyn id="play-button" class="carousel-slide-right" insert="image-slider">Play</dyn>
</div>
```

Search Example

```
<!-- Create a search interface -->
<search id="content-search" use-id="content1;content2;content3;" input-width="200px" input-height="30px"

<!-- Content areas to search -->
<div id="content1">Content 1</div>
<div id="content2">Content 2</div>
<div id="content3">Content 3</div>
```

CSV Data Example

```
<!-- Display CSV data as a table -->
<csv id="data-table" sources="data.csv" csv-as="table" sort="name:csv-asc" page-size="10">
  <template>
    <thead>
      <tr>
        <th>Name</th>
        <th>Email</th>
        <th>Phone</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>{{name}}</td>
        <td>{{email}}</td>
        <td>{{phone}}</td>
      </tr>
    </tbody>
  </template>
</csv>
```

Modal Example

```
<!-- Button to show modal -->
<dyn id="show-modal" modal="modal.json:modal-container">Show Modal</dyn>

<!-- Modal container -->
<div id="modal-container"></div>
```

Tree View Example

```
<!-- Load and display a tree view -->
<pipe id="tree-loader" class="tree-view" ajax="tree.json" insert="tree-container"></pipe>

<!-- Tree container -->
<div id="tree-container"></div>
```

API Reference

Global Variables

- `PAGE_NONCE` : Security nonce for Content Security Policy

Event Handlers

- `document.addEventListener("DOMContentLoaded", function () {...})` : Initializes dotPipe when the DOM is loaded

Component APIs

- `window.columnsComponent` : API for columns component
- `refreshColumn(columnsId, columnIndex)` : Refreshes a specific column
- `updateColumnContent(columnsId, columnIndex, newContent)` : Updates content in a column
- `loadColumnPage(columnsId, columnIndex, pageUrl)` : Loads a new page into a column
- `window.refreshComponent` : API for refresh component
- `refreshTarget(targetId, pageUrl)` : Refreshes content in a target element
- `refreshTargets(targetSpec)` : Refreshes multiple targets

Custom Events

- `columnContentLoaded` : Fired when column content is loaded
- `refreshStarted` : Fired when content refresh starts
- `refreshCompleted` : Fired when content refresh completes
- `refreshFailed` : Fired when content refresh fails
- `csvLoaded` : Fired when CSV data is loaded
- `csvMoreDataLoaded` : Fired when more CSV data is loaded
- `searchPerformed` : Fired when search is performed

Security Functions

- `generateNonce()` : Generates a security nonce for Content Security Policy
- `sha256(message)` : Computes SHA-256 hash of a message

This manual provides a comprehensive reference for dotPipe.js, covering all tags, attributes, classes, helper functions, flow patterns, and examples. Use it as a guide to leverage the full capabilities of the dotPipe library in your web applications.